ASSIGNMENT_01 JULIA

NISHANT KUMAR 22CE01010

GITHUB: https://github.com/NishantKumar-1529

Ques 1

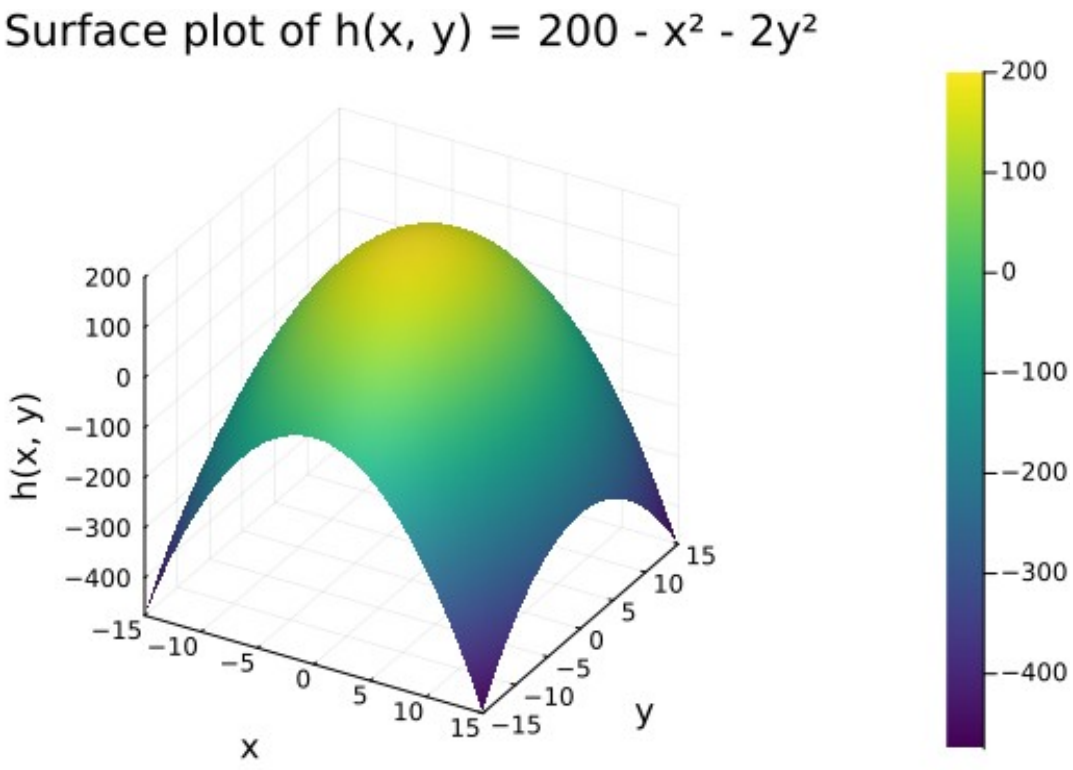
Consider that the height of a hill is described by the given scalar field as

$$h(x,y)=200-x^2-2y^2$$

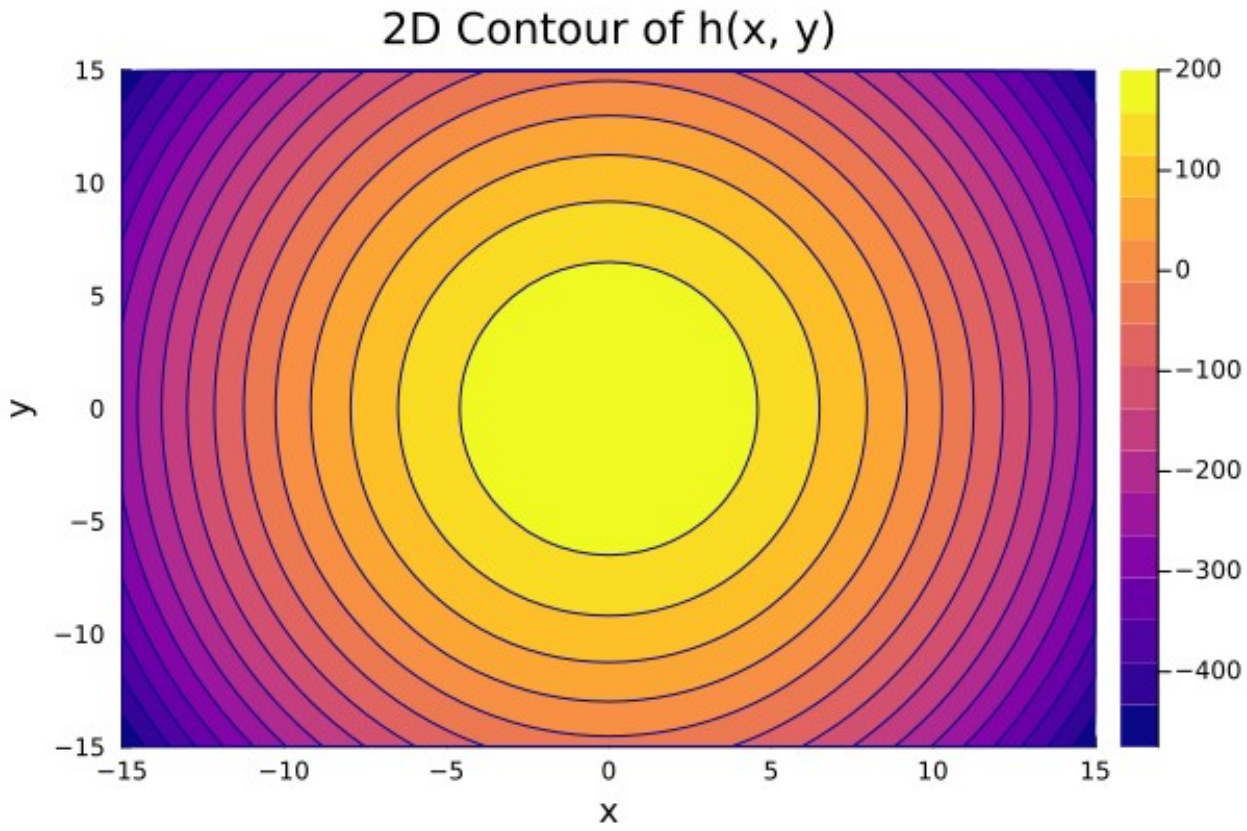
(a) Plot the given scalar field as both a three-dimensional (3D) surface plot and a two-dimensional (2D) contour plot using Julia. (You may use the package `Plots.jl` for plotting.)

```
import Pkg
Pkg.add("Plots")
Pkg.add("CalculusWithJulia")

using Plots

f(a, b) = 200 - a^2 - 2*b^2

x_vals = collect(-15:0.5:15)
y_vals = collect(-15:0.5:15)

z = [f(a, b) for a in x_vals, b in y_vals]

surface(x_vals, y_vals, z,
    xlabel="x",
    ylabel="y",
    zlabel="h(x, y)",
    title="Surface plot of h(x, y) = 200 - x² - 2y²",
    color=:viridis
)
```

```
   Resolving package versions...
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Project.toml`
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Manifest.toml`
   Resolving package versions...
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Project.toml`
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Manifest.toml`
```




```
h(x, y) = 200 - x^2 - 2y^2

x = -15:0.5:15
y = -15:0.5:15

z = [h(a, b) for a in x, b in y]

contour(x, y, z,
    title = "2D Contour of h(x, y)",
    xlabel = "x",
    ylabel = "y",
    fill = true,
    color = :plasma
)
```



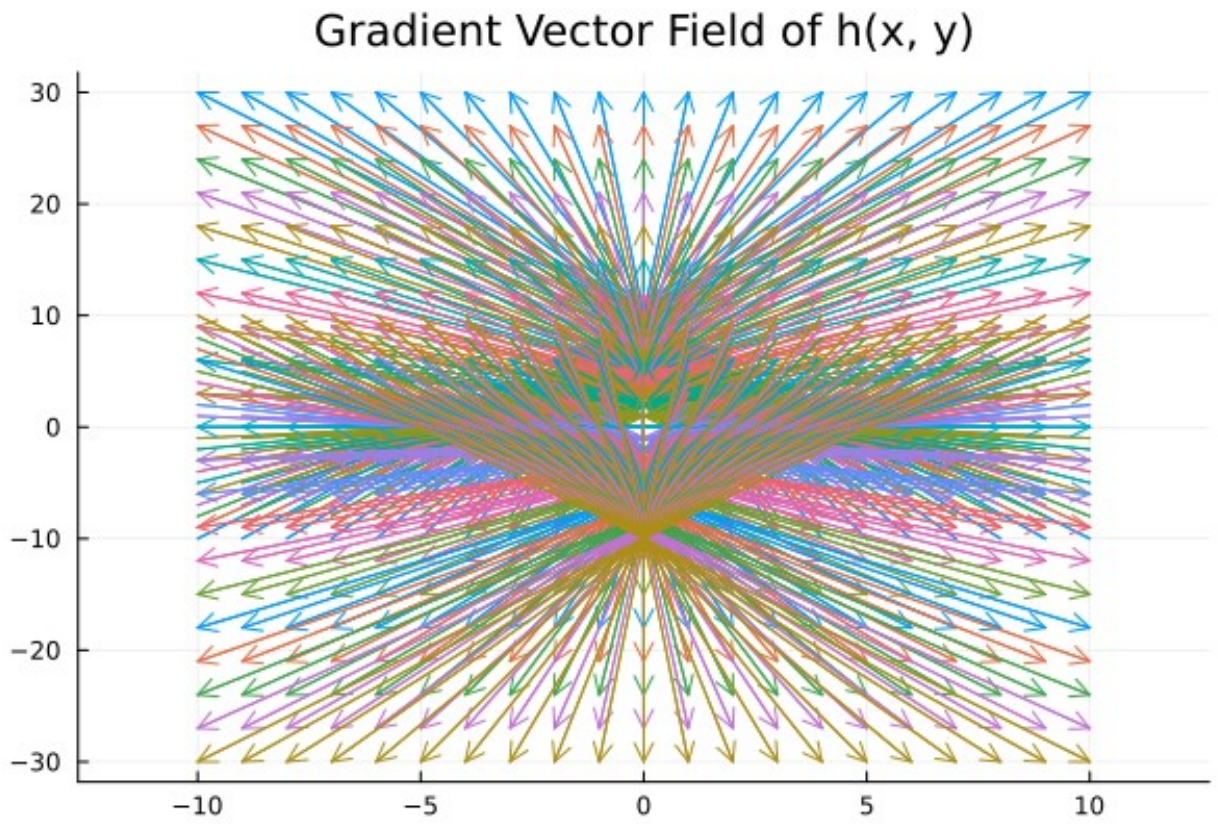
(b)Plot the gradient of the scalar field using the automatic gradient calculation tool available in Julia (you may use the package called CalculusWithJulia.jl).

```
using CalculusWithJulia

f(v) = 200 - v[1]^2 - 2*v[2]^2
grad_f = gradient(f)
grad_at_point = grad_f([3, 5])
println(grad_at_point)
```

```
[-6, -20]
```

```
using Plots

grad_h(x, y) = (-2x, -4y)

x_vals = -10:1:10
y_vals = -10:1:10

X_grid = [x for x in x_vals, y in y_vals]
Y_grid = [y for x in x_vals, y in y_vals]
U_grid = [-2x for x in x_vals, y in y_vals]
V_grid = [-4y for x in x_vals, y in y_vals]

quiver(X_grid, Y_grid, quiver=(U_grid, V_grid),
```

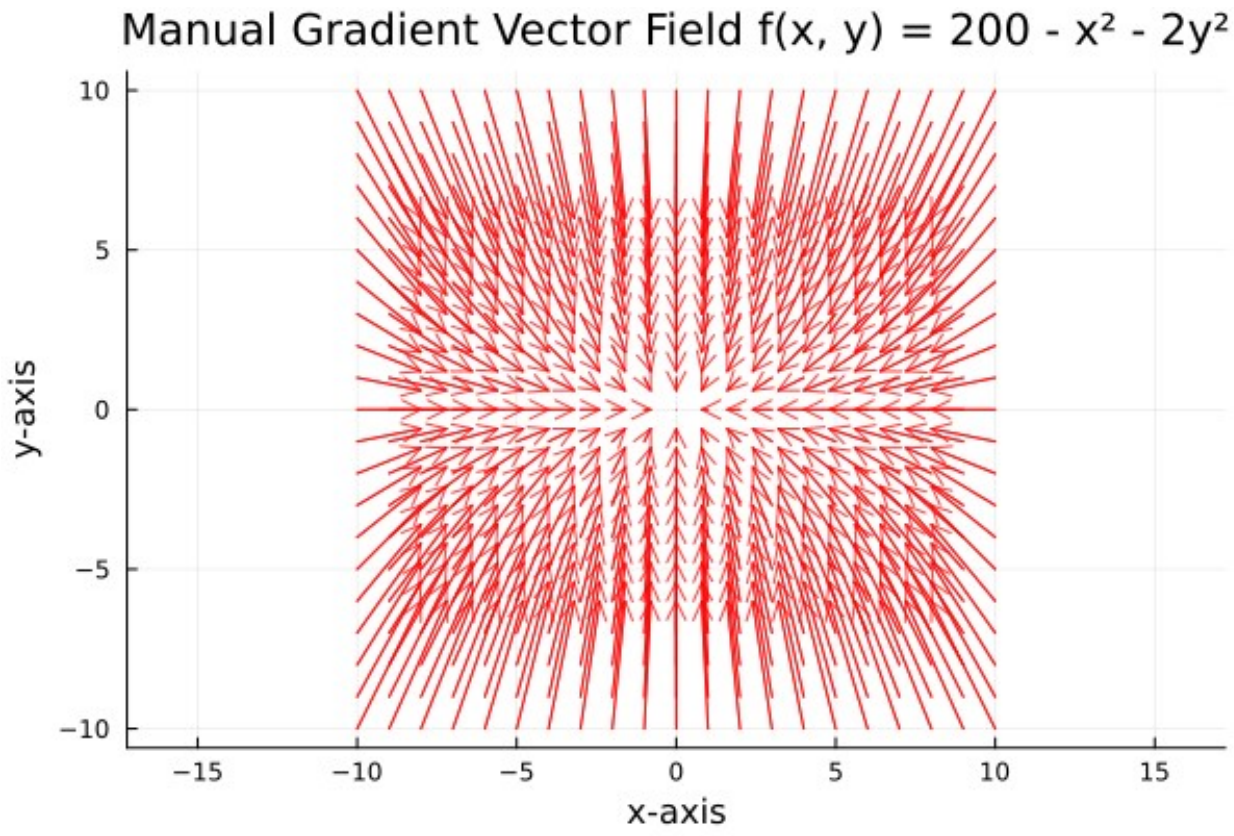
```
        aspect_ratio=0.25,
        title="Gradient Vector Field of h(x, y)")
```



(c) Determine the gradient vector and plot the obtained gradient vector field (you may use the package called Plots.jl or CalculusWithJulia.jl)

```
using CalculusWithJulia
using Plots

f(v) = 200 - v[1]^2 - 2*v[2]^2
fx(x, y) = -2x
fy(x, y) = -4y

x_vals = -10.0:1.0:10.0
y_vals = -10.0:1.0:10.0

X_grid = [x for x in x_vals, y in y_vals]
Y_grid = [y for x in x_vals, y in y_vals]

U_grid = [fx(x, y) for x in x_vals, y in y_vals]
V_grid = [fy(x, y) for x in x_vals, y in y_vals]

scale_factor = 0.10
U_grid .*= scale_factor
```

```
V_grid .*= scale_factor

quiver(X_grid, Y_grid,
    quiver = (U_grid, V_grid),
    xlabel = "x-axis",
    ylabel = "y-axis",
    title = "Manual Gradient Vector Field f(x, y) = 200 - x² - 2y²",
    linealpha = 0.75,
    arrowsize = 0.3,
    aspect_ratio = :equal,
    legend = false,
    color = :red
)
```



Ques 2

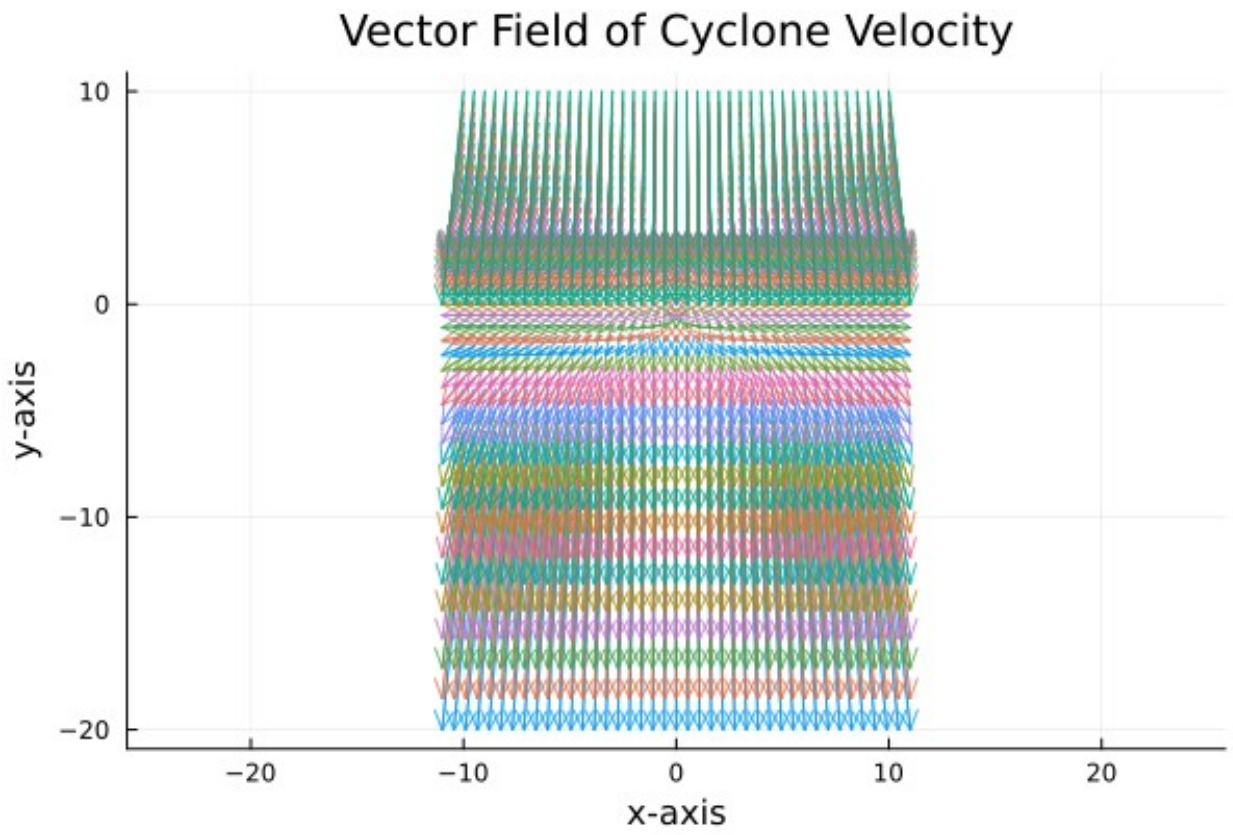
Consider a cyclone in the northern hemisphere described by the velocity vector field of the wind:

$$v(x,y)=xe_1-y^2e_2$$

where (x) and (y) are the coordinates in the horizontal plane, and (\mathbf{e_1}) and (\mathbf{e_2}) are unit vectors in the x- and y-directions, respectively.

(a) Plot the given vector field in Julia. (You may use the package called `Plots.jl` or `CalculusWithJulia.jl`.)

```
using Plots

function velocity_vector(x, y)
    return [x, -y^2]
end

x_vals = -10:0.5:10
y_vals = -10:0.5:10

U_grid = [velocity_vector(x, y)[1] for x in x_vals, y in y_vals]
V_grid = [velocity_vector(x, y)[2] for x in x_vals, y in y_vals]

X_grid = [x for x in x_vals, y in y_vals]
Y_grid = [y for x in x_vals, y in y_vals]

scale_factor = 0.10
U_grid .*= scale_factor
V_grid .*= scale_factor

quiver(X_grid, Y_grid,
       quiver = (U_grid, V_grid),
       title = "Vector Field of Cyclone Velocity",
       xlabel = "x-axis",
       ylabel = "y-axis",
       linealpha = 0.75,
       arrowsize = 0.3,
       aspect_ratio = :equal
)
```



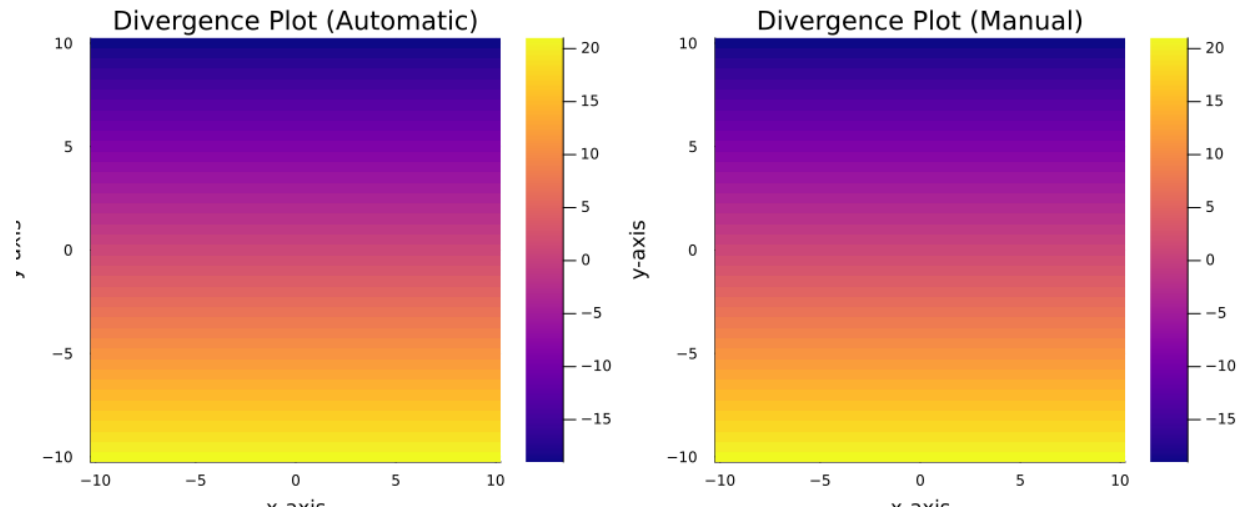

```
using Pkg
Pkg.add("SymPy")
```

```
   Resolving package versions...
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Project.toml`
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Manifest.toml`
```

(b) Plot the divergence of the vector field using automatic divergence calculation available in the Julia package called CalculusWithJulia.jl. Also, determine the divergence using the detailed calculation and plot the same.Compare both plots and verify the results.

```
using Plots
using CalculusWithJulia

# Function for velocity vector
function velocity_vector(x, y)
    return [x, -y^2]
end

# Divergence using automatic gradient
div_auto(x, y) = divergence(u -> velocity_vector(u[1], u[2]), [x, y])
```

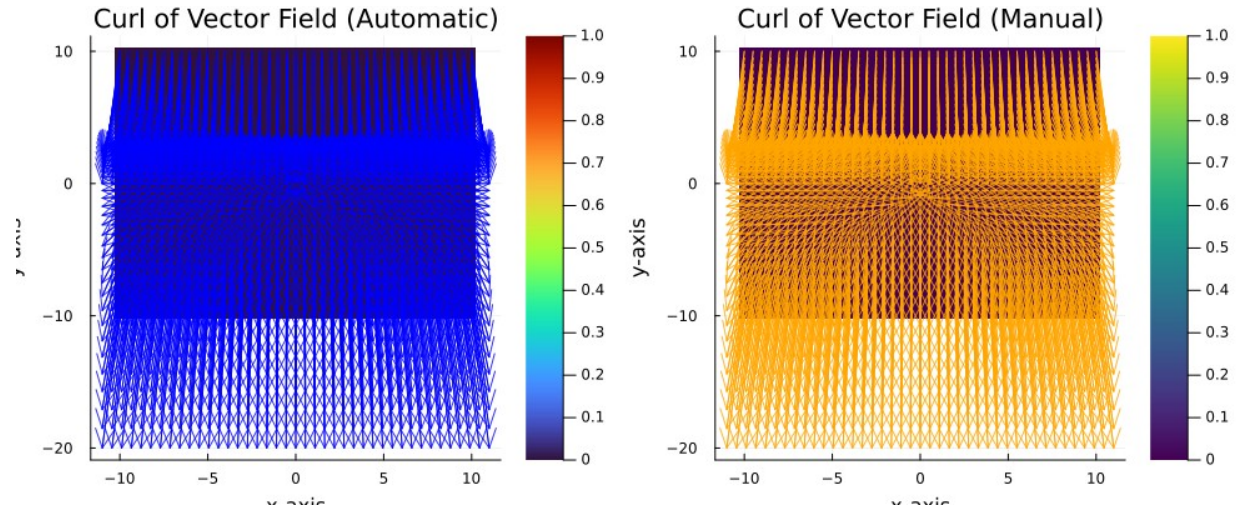
```
# Grid
x_range = -10:0.5:10
y_range = -10:0.5:10

# Color gradient
color_grad = :plasma

# Automatic divergence heatmap
p1 = heatmap(x_range, y_range, div_auto,
    xlabel = "x-axis",
    ylabel = "y-axis",
    zlabel = "Divergence",
    c = color_grad,
    title = "Divergence Plot (Automatic)"
)

# Divergence computed manually
div_manual(x, y) = gradient(u -> velocity_vector(u[1], u[2])[1], [x,
y])[1] +
                   gradient(u -> velocity_vector(u[1], u[2])[2], [x,
y])[2]

# Manual divergence heatmap
p2 = heatmap(x_range, y_range, div_manual,
    xlabel = "x-axis",
    ylabel = "y-axis",
    zlabel = "Divergence (Manual)",
    c = color_grad,
    title = "Divergence Plot (Manual)"
)

# Combine plots side by side
plot(p1, p2, layout = (1, 2), size = (1000, 400))
```



(c)Determine the curl of the vector field using automatic curl calculation available in the Julia package CalculusWithJulia.jl. Also, determine the curl using detailed calculation and plot the same. Compare both plots and verify the results.

```
using Plots
using CalculusWithJulia

function velocity_vector(x, y)
    return [x, -y^2]
end

x_vals = -10:0.5:10
y_vals = -10:0.5:10

U = [velocity_vector(x, y)[1] for x in x_vals, y in y_vals]
V = [velocity_vector(x, y)[2] for x in x_vals, y in y_vals]
X = [x for x in x_vals, y in y_vals]
Y = [y for x in x_vals, y in y_vals]

arrow_scale = 0.1
U .*= arrow_scale
V .*= arrow_scale

curl_auto(x, y) = curl(u -> velocity_vector(u[1], u[2]), [x, y])
curl_manual(x, y) = gradient(u -> velocity_vector(u[1], u[2])[2], [x,
y])[1] -
                    gradient(u -> velocity_vector(u[1], u[2])[1], [x,
y])[2]

p1 = heatmap(x_vals, y_vals, curl_auto,
             color = :turbo,
             title = "Curl of Vector Field (Automatic)",
             xlabel = "x-axis",
             ylabel = "y-axis")

quiver!(p1, X, Y, quiver = (U, V),
        color = :blue,
        linealpha = 0.8,
        arrowsize = 0.3)

p2 = heatmap(x_vals, y_vals, curl_manual,
             color = :viridis,
             title = "Curl of Vector Field (Manual)",
             xlabel = "x-axis",
             ylabel = "y-axis")

quiver!(p2, X, Y, quiver = (U, V),
        color = :orange,
        linealpha = 0.8,
        arrowsize = 0.3)
```

```
plot(p1, p2, layout = (1, 2), size = (1000, 400))
```



Ques 3

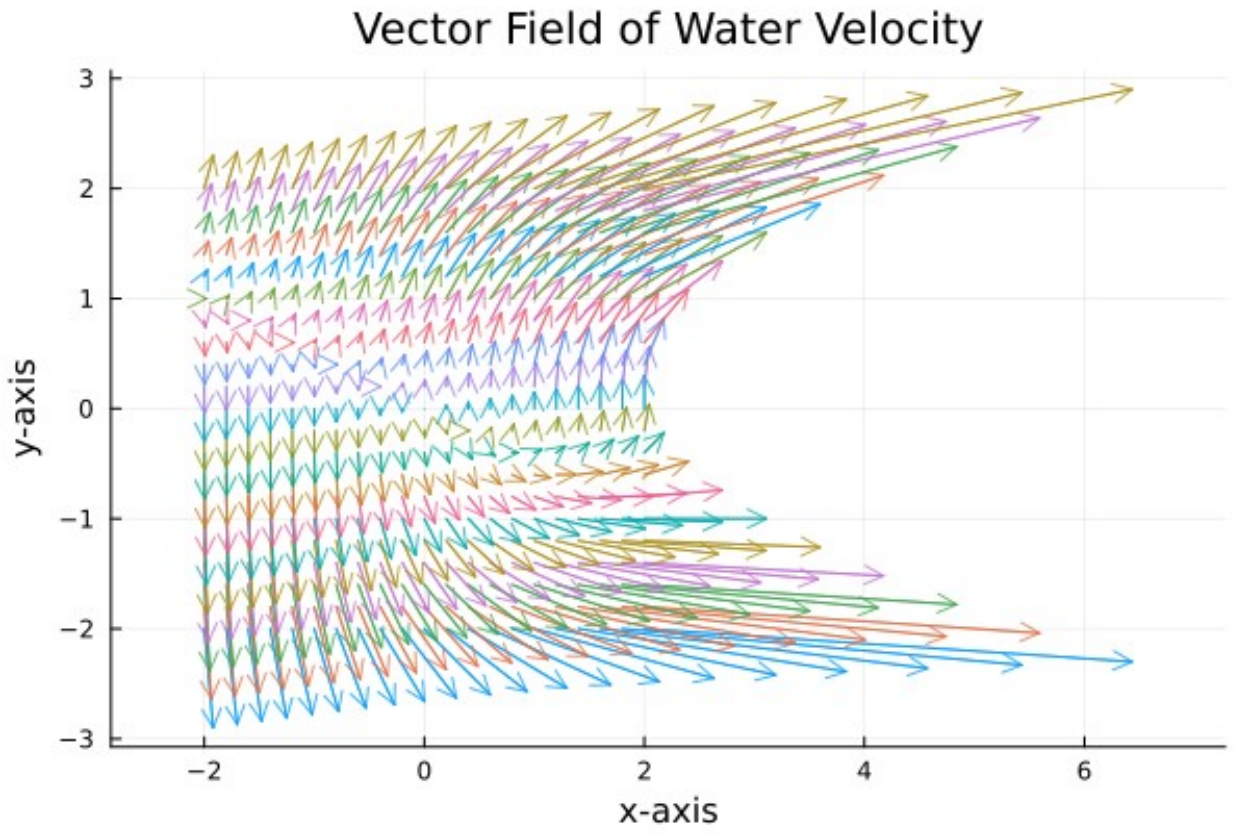
Consider that the velocity of water particles in a river is described by the vector field

$$f = e^x y^2 e_1 + (x + 2y) e_2$$

(a) Plot the given vector field in Julia. (You may use the package called `Plots.jl` or `CalculusWithJulia.jl`.)

```
using Plots

function velocity_vector(x, y)
    return [exp(x) * y^2, x + 2*y]
end

x_vals = -2:0.2:2
y_vals = -2:0.2:2

U_grid = [velocity_vector(x, y)[1] for x in x_vals, y in y_vals]
V_grid = [velocity_vector(x, y)[2] for x in x_vals, y in y_vals]

X_grid = [x for x in x_vals, y in y_vals]
Y_grid = [y for x in x_vals, y in y_vals]

arrow_scale = 0.15
U_grid .*= arrow_scale
V_grid .*= arrow_scale

quiver(X_grid, Y_grid,
       quiver = (U_grid, V_grid),
       title = "Vector Field of Water Velocity",
```

```
        xlabel = "x-axis",
        ylabel = "y-axis",
        linealpha = 0.75,
        arrowsize = 0.3,
        aspect_ratio = :equal
)
```



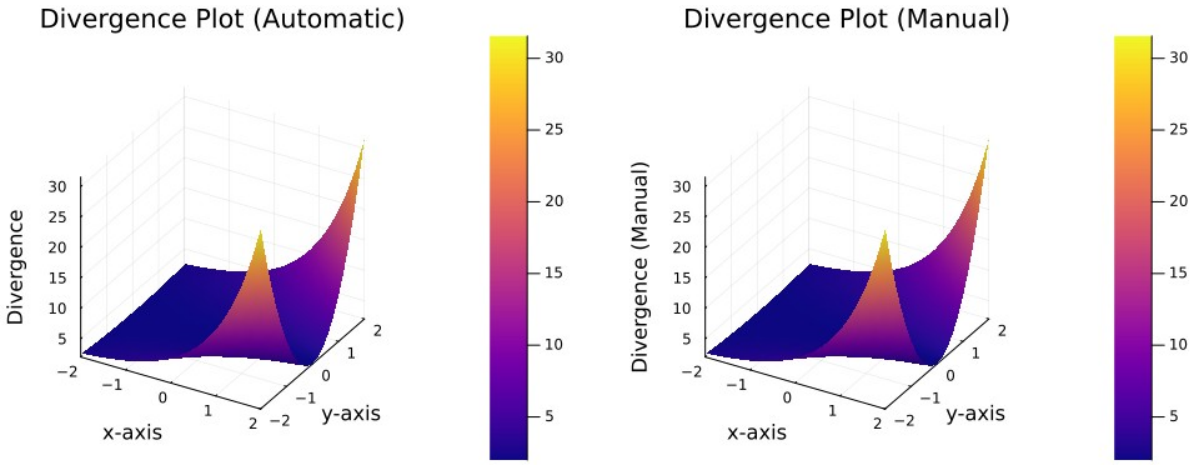
(b) Plot the divergence of the vector field using automatic divergence calculation available in the Julia package called CalculusWithJulia.jl. Also, determine the divergence using the detailed calculation and plot the same. Compare both plots and verify the results.

```
using Pkg
Pkg.add("SymPy")
```

```
   Resolving package versions...
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Project.toml`
  No Changes to `C:\Users\nisha\.julia\environments\v1.11\
Manifest.toml`
```

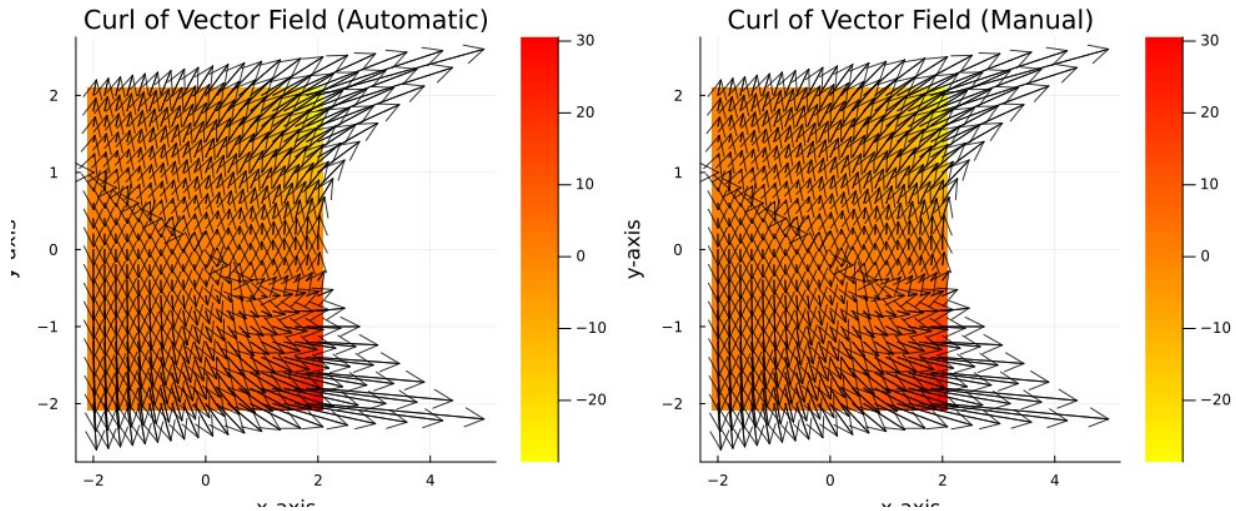
```
using Plots
using CalculusWithJulia

function velocity_vector(x, y)
```

```
    return [exp(x) * y^2, x + 2*y]
end

div_auto(x, y) = divergence(u -> velocity_vector(u[1], u[2]), [x, y])
div_manual(x, y) = gradient(u -> velocity_vector(u[1], u[2])[1], [x,
y])[1] +
                   gradient(u -> velocity_vector(u[1], u[2])[2], [x,
y])[2]

x_vals = -2:0.2:2
y_vals = -2:0.2:2
color_grad = :plasma

p1 = surface(x_vals, y_vals, div_auto,
             xlabel = "x-axis",
             ylabel = "y-axis",
             zlabel = "Divergence",
             c = color_grad,
             title = "Divergence Plot (Automatic)"
)

p2 = surface(x_vals, y_vals, div_manual,
             xlabel = "x-axis",
             ylabel = "y-axis",
             zlabel = "Divergence (Manual)",
             c = color_grad,
             title = "Divergence Plot (Manual)"
)

plot(p1, p2, layout = (1, 2), size = (1000, 400))
```



(c) Determine the curl of the vector field using automatic curl calculation available in the Julia package CalculusWithJulia.jl. Also, determine the curl using detailed calculation and plot the same. Compare both plots and verify the results

```
using Plots
using CalculusWithJulia

function velocity_vector(x, y)
    return [exp(x) * y^2, x + 2*y]
end

x_vals = -2:0.2:2
y_vals = -2:0.2:2

U = [velocity_vector(x, y)[1] for x in x_vals, y in y_vals]
V = [velocity_vector(x, y)[2] for x in x_vals, y in y_vals]
X = [x for x in x_vals, y in y_vals]
Y = [y for x in x_vals, y in y_vals]

arrow_scale = 0.1
U .*= arrow_scale
V .*= arrow_scale

curl_auto(x, y) = curl(u -> velocity_vector(u[1], u[2]), [x, y])
curl_manual(x, y) = gradient(u -> velocity_vector(u[1], u[2])[2], [x,
y])[1] -
                    gradient(u -> velocity_vector(u[1], u[2])[1], [x,
y])[2]

color_grad = cgrad([:yellow, :red])

p1 = heatmap(x_vals, y_vals, curl_auto,
             color = color_grad,
             title = "Curl of Vector Field (Automatic)",
             xlabel = "x-axis",
             ylabel = "y-axis")

quiver!(p1, X, Y, quiver = (U, V),
        color = :black,
        linealpha = 0.8,
        arrowsize = 0.3)

p2 = heatmap(x_vals, y_vals, curl_manual,
             color = color_grad,
             title = "Curl of Vector Field (Manual)",
             xlabel = "x-axis",
             ylabel = "y-axis")

quiver!(p2, X, Y, quiver = (U, V),
        color = :black,
        linealpha = 0.8,
        arrowsize = 0.3)

plot(p1, p2, layout = (1, 2), size = (1000, 400))
```



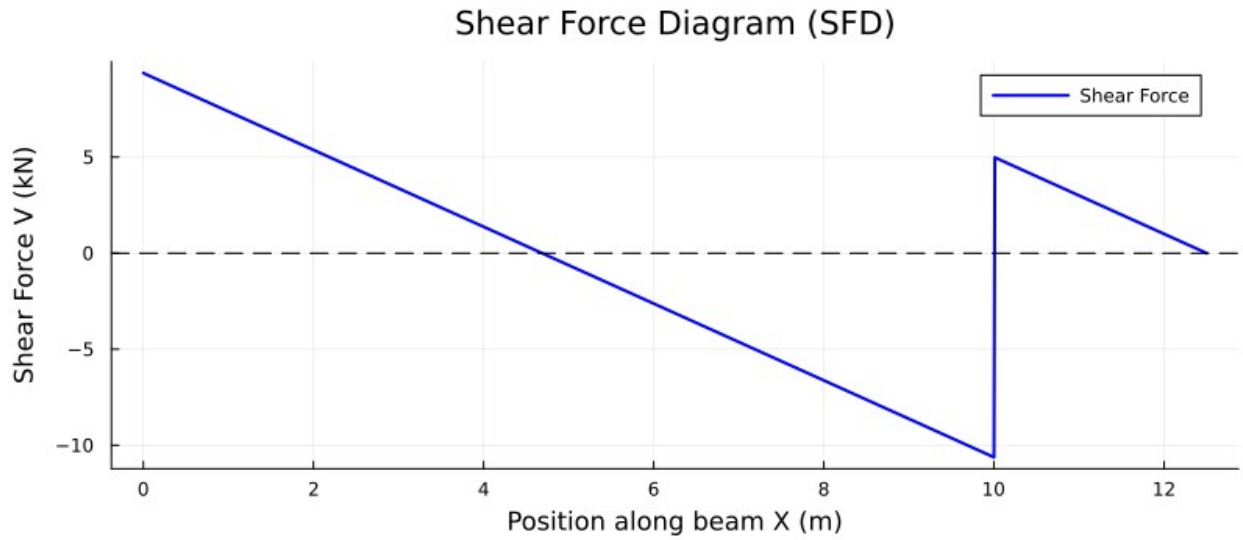
Ques 4

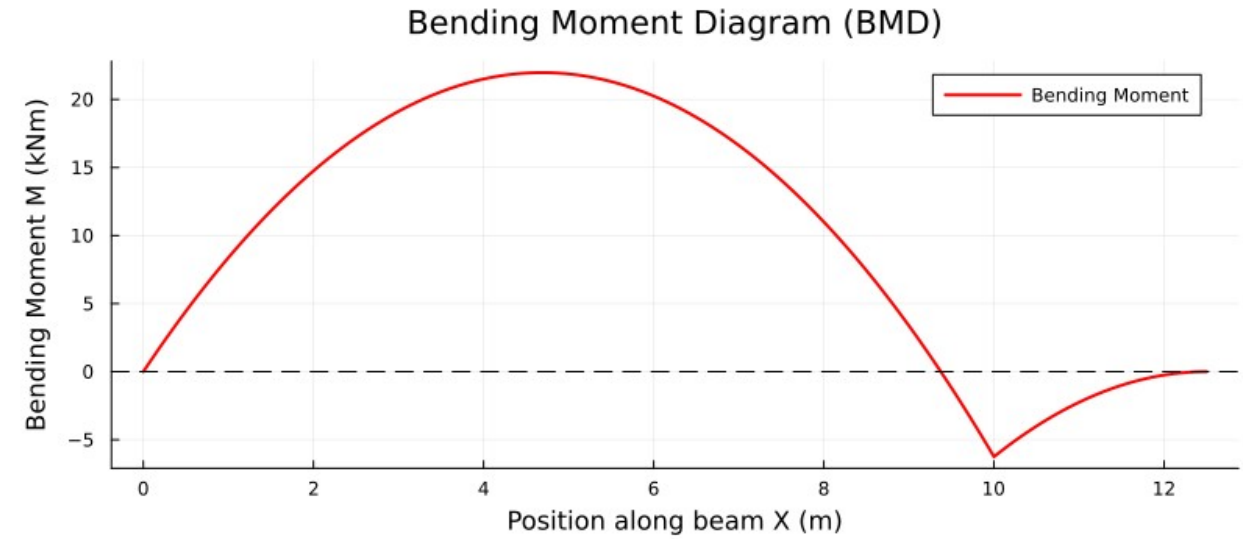
Write a Julia code for the solution of the beam problem shown in Fig. 1.
Plot the bending moment diagram (BMD) and shear force diagram (SFD).
Your code should be generic enough to take any value for the input variables `l` and `q`.

Beam Problem

```
using Plots

l = 10.0
q = 2.0

total_load = q * (1.25 * l)
R_B = (25 * q * l / 32)
R_A = total_load - R_B

println("Reaction at A (R_A): $R_A kN")
println("Reaction at B (R_B): $R_B kN")

function shear_force(x)
    if 0 <= x <= l
        return (15 * q * l / 32 - q * x)
    else
        return (40 * q * l / 32 - q * x)
    end
end

function bending_moment(x)
    if 0 <= x <= l
        return 15 * q * l * x / 32 - (q * x^2) / 2
    else
        return 15 * q * l / 32 * x + (25 * q * l / 32) * (x - l) - (q
* x^2) / 2
    end
```

```
end

total_length = 1.25 * l
x_vals = 0:0.01:total_length

V_vals = shear_force.(x_vals)
M_vals = bending_moment.(x_vals)

sfd_plot = plot(x_vals, V_vals,
    label="Shear Force",
    title="Shear Force Diagram (SFD)",
    xlabel="Position along beam X (m)",
    ylabel="Shear Force V (kN)",
    lw=2,
    color=:blue,
    legend=:topright
)
hline!([0], color=:black, linestyle=:dash, label="")

bmd_plot = plot(x_vals, M_vals,
    label="Bending Moment",
    title="Bending Moment Diagram (BMD)",
    xlabel="Position along beam X (m)",
    ylabel="Bending Moment M (kNm)",
    lw=2,
    color=:red,
    legend=:topright
)
hline!([0], color=:black, linestyle=:dash, label="")

plot(sfd_plot, bmd_plot, layout=(2, 1), size=(800, 700))

Reaction at A (R_A): 9.375 kN
Reaction at B (R_B): 15.625 kN
```





Calculations of Support Reactions

For a simply supported **overhang beam** of span (l) and overhung part of (0.25l), subjected to a uniformly distributed load (q):

Equilibrium Equations

$$\sum F_y = 0 \Rightarrow R_A + R_B - \frac{5}{4} q l = 0$$

$$R_A + R_B = \frac{5}{4} q l$$

Moment About A

$$\sum M_A = 0 \Rightarrow R_B \cdot l - q\,l \cdot \frac{(1.25\,l)}{2} = 0$$

$$R_B = \frac{3}{25} q\,l$$

Reactions

$$R_A = \frac{5}{4} q\,l - \frac{3}{25} q\,l = \frac{32}{15} q\,l$$

$$R_B = \frac{3}{25} q\,l$$

Shear Force and Bending Moment Equations

Let (x) be the distance measured from the left support (A).

For (0 \le x \le l) (within the main beam):

$$V(x) = R_A - q\,x$$

$$M(x) = R_A\,x - \frac{q\,x^2}{2}$$

For (x > l) (overhanging part):

$$V(x) = R_A + R_B - q\,x$$

$$M(x) = R_A\,x + R_B(x - l) - \frac{q\,x^2}{2}$$

Ques 5

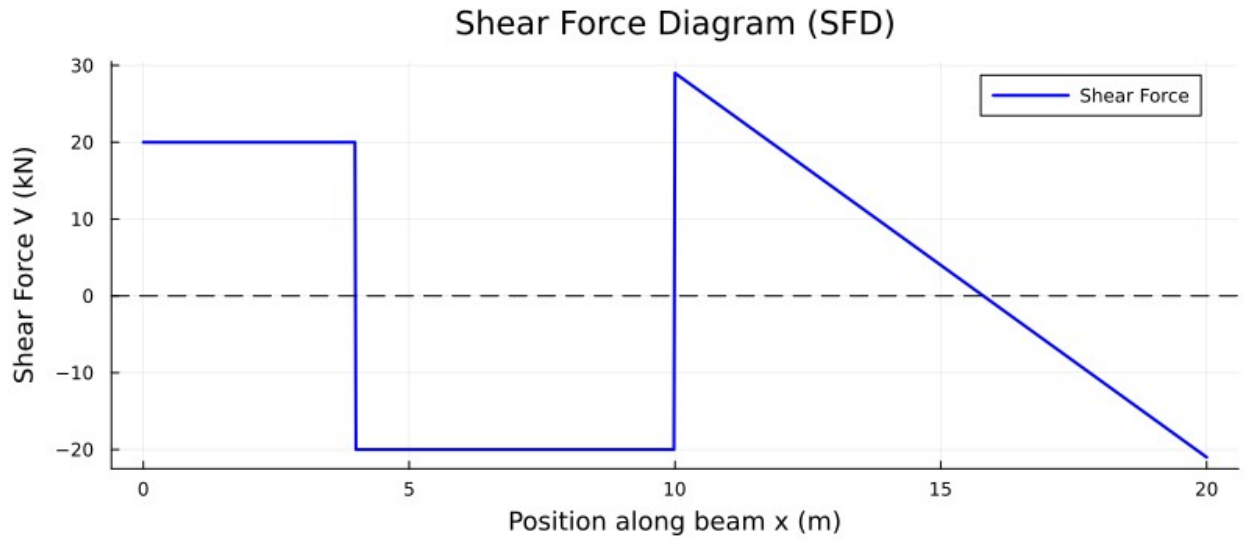
Write a Julia code for the solution of the beam problem shown in Fig. 2. Plot the BMD and SFD. Your code should be generic enough to take any value for the input variables l and q.

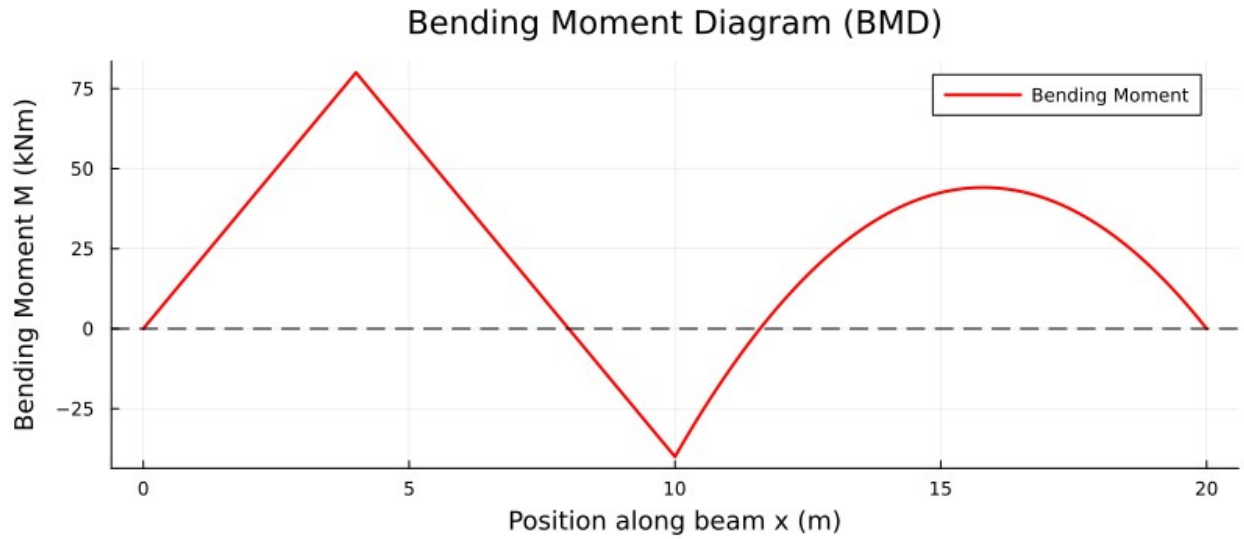
Beam Problem

```julia
using Plots

l = 10.0
q = 5.0

R_A = (0.8 * q * l * (0.4 * l)) / (0.8 * l)
R_C = (R_A*l - (0.8*q*l)*(0.6*l) + (q*l)*(0.5*l)) / l
R_B = (0.8 * q * l) + (q * l) - R_A - R_C

println("Reaction at A: $R_A kN")
println("Reaction at B: $R_B kN")
println("Reaction at C: $R_C kN")
println("Net reactions: $(R_A + R_B + R_C) kN")
println("Total load: $((0.8*q*l) + (q*l)) kN")

function shear_force(x)
    if 0 <= x < 0.4*l
        return R_A
    elseif 0.4*l <= x < l
        return R_A - (0.8 * q * l)
    elseif l <= x <= 2*l
        return R_A - (0.8 * q * l) + R_B - q * (x - l)
    else
        return 0.0
    end
end

function bending_moment(x)
    if 0 <= x < 0.4*l
        return R_A * x
    elseif 0.4*l <= x < l
        return R_A * x - (0.8 * q * l) * (x - 0.4 * l)
    elseif l <= x <= 2*l
        return R_A * x - (0.8 * q * l) * (x - 0.4 * l) + R_B * (x - l)
- q * (x - l)^2 / 2
    else
        return 0.0
    end
end

total_length = 2 * l
x_vals = 0:total_length/1000:total_length

V_vals = shear_force.(x_vals)
M_vals = bending_moment.(x_vals)

sfd_plot = plot(x_vals, V_vals,
    label="Shear Force",
    title="Shear Force Diagram (SFD)",
    xlabel="Position along beam x (m)",
```

```
    ylabel="Shear Force V (kN)",
    lw=2,
    color=:blue,
    legend=:topright
)
hline!([0], color=:black, linestyle=:dash, label="")

bmd_plot = plot(x_vals, M_vals,
    label="Bending Moment",
    title="Bending Moment Diagram (BMD)",
    xlabel="Position along beam x (m)",
    ylabel="Bending Moment M (kNm)",
    lw=2,
    color=:red,
    legend=:topright
)
hline!([0], color=:black, linestyle=:dash, label="")

plot(sfd_plot, bmd_plot, layout=(2, 1), size=(800, 700))
```

```
Reaction at A: 20.0 kN
Reaction at B: 49.0 kN
Reaction at C: 21.0 kN
Net reactions: 90.0 kN
Total load: 90.0 kN
```





Calculations of Support Reactions

For the beam under a uniformly distributed load (q):

Equilibrium Equation

$$\sum F_y = 0 \Rightarrow R_A + R_B + R_C - 1.8\,q\,l = 0$$

$$R_A + R_B + R_C = 1.8\,q\,l \quad (1)$$

Moment About D (First Equation)

$$\sum M_D = 0 \Rightarrow R_A(0.8l) - (0.8ql)(0.4l) = 0$$

$$R_A = 0.4ql \quad (2)$$

Moment About D (Second Equation)

$$\sum M_D = 0 \Rightarrow R_B(0.2l) + R_C(1.2l) - 0.7ql = 0$$

$$R_B(0.2l) + R_C(1.2l) = 0.7ql \quad (3)$$

Solving Equations (1), (2), and (3):

$$R_A = 0.4ql$$

$$R_B = 0.98ql$$

$$R_C = 0.42ql$$

Shear Force and Bending Moment Equations

Let (x) be the distance measured from the left support (A).

For (0 ≤ x ≤ 0.4l):

$$V(x) = R_A$$

$$M(x) = R_A x$$

For (0.4l ≤ x ≤ 0.8l):

$$V(x) = R_A - 0.8ql$$

$$M(x) = R_A x - 0.8ql(x - 0.4l)$$

For (0.8l ≤ x ≤ l):

$$V(x) = R_A - 0.8\,q\,l$$

$$M(x) = R_A\,x - P\,(x - 0.4\,l)$$

For (x > l):

$$V(x) = R_A - P + R_B - q\,(x - l)$$

$$M(x) = R_A\,x - 0.8\,q\,l\,(x - 0.4\,l) + R_B\,(x - l) - \frac{q\,(x - l)^2}{2}$$